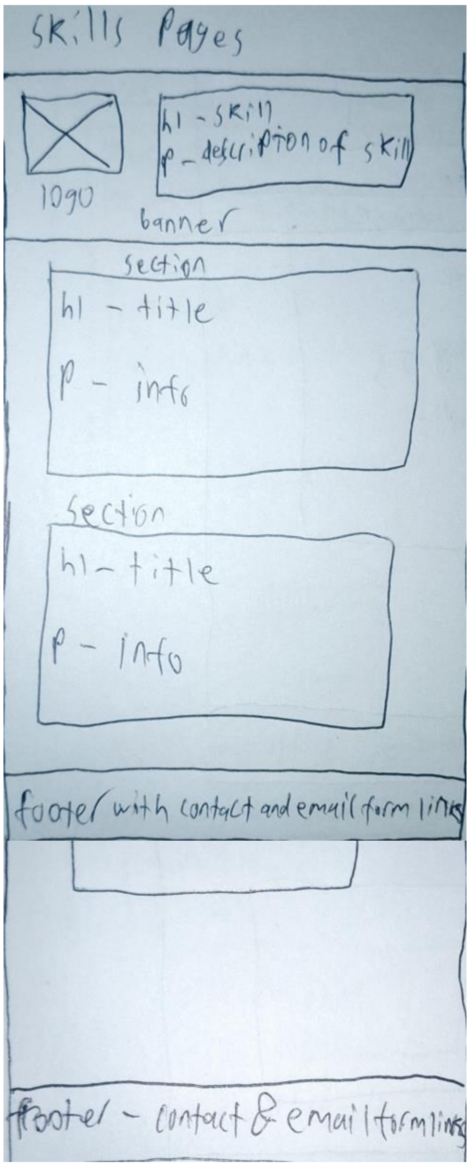
Access website hosted on GitHub [here.]

Storyboards

Skills Pages



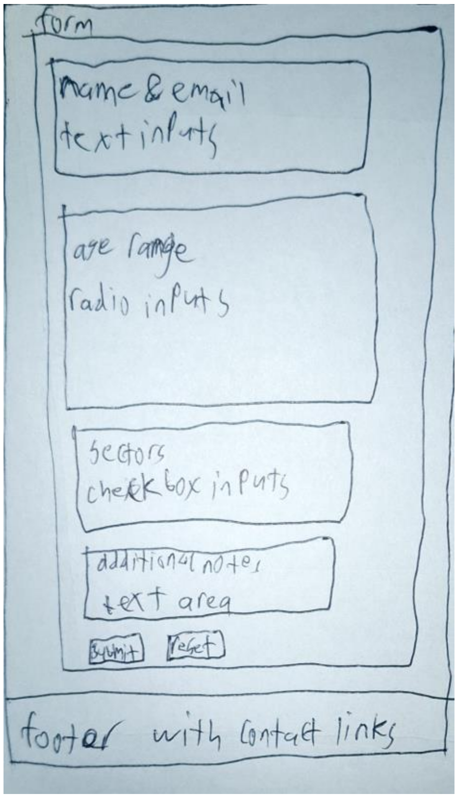
The structure for the fast learning, time management, and programming skills pages use a similar structure. They keep the same header and footer as the previous pages. The main content in the middle is split into sections, each with their own title and information.

JavaScript Examples

The JavaScript examples page uses the same header and footer as the other pages. The main content contains a horizontal navigation bar with buttons that change the currently displayed example. Most of the page is taken up by the iframe that is used to display the current JavaScript example.

Email Form

The email newsletter sign up page keeps the same footer as the rest of the page except for the link to the newsletter sign up. I chose to remove the header for this page since it was unnecessary, and it would keep more room for the form. The form is split into



different 'fieldset' elements that split up the different sections of the form to make it easier to read.

Design

Page Design

The design of each page is similar to the design of the previous assessments; each page has an identical header and footer with the main content in-between, all structured using a CSS flexbox with the 'column' flex-direction property value.

I used a light gray background color and black font color across the entire page to make the text easy to read.

I also used a light green on black color scheme in multiple sections of the website to fit the video game development company theme I chose for the fictional 'Game Software' company.

Additionally, I have used more vibrant colors for links to differentiate them from other texts while also keeping them visible with lighter backgrounds.

Interface Design

The interface design of the email newsletter sign up form uses 'fieldset' and 'legend' elements to group together and title the different inputs to separate each section of the form.

The interface design of the JavaScript Examples page is achieved by using an iframe with the source as a URL to the hosted website of each example.

Navigation Design

Navigation between the major pages within the website has been kept the same as the previous assessments where a horizontal navigation bar is created using 'ul' and 'li' elements and styled with CSS. I have made changes to these navigation bars to make them identical across each page and using different colors to show which page the user is currently on.

Navigation to the new skills pages is available through links in the resume page. I made this decision because the skills I talk about in the new pages have already been listed in the resume page, and all that needed to change was making them anchor elements.

Navigation to the email sign up page is accessible through the footers. Each page has an identical footer with a link to the email form page (except for the email form page itself, which does not need to navigate to itself).

New Content

Skills Pages

Additional pages detailing some of my skills that can be important for a workplace have been added to follow the assignment requirements. The new pages detail skills and experiences with time management, fast and adaptive learning, programming, and JavaScript. The new pages are all accessible through internal links in the resume page.

Most of the pages provide information to employers on my experiences and examples of each skill through text content split into sections. The JavaScript skills page contains an iframe with URLs to some examples of my JavaScript experiences hosted on GitHub.

Email Newsletter Form

A new web page with a form has been added to the website as required by the assessment requirements. The form contains text inputs for the users first name, last name, and email all with a character maximum of 60, a radio input group for the age range of the user, a checkbox input group for the sectors that the user wants to sign up for, and a text area with a minimum size of 60 columns and 3 rows for any additional notes the user wants to input.

When clicking the submit button, a JavaScript function is called to first check if the first name, last name, email, and age range inputs have values. After verifying this, the function adds all the values to a string names variable names ‘out’ and alerts the string. I decided to have the values be output as an alert rather than outputting it as part of the page because doing so would require using the ‘preventDefault()’ method which would also prevent submitting the form data with the ‘post’ action method that is required in the outline of the assessment. After outputting the values, the form window will be closed because it will no longer be needed by the user.

The form also contains a reset button that removes the current inputs that the user has made and makes all input elements empty again.

The page for the email form is available from every other page on the website through an internal link in the footer of each page.

Feedback Changes

I have made changes to the website based on the feedback given to me from my assessment 3 submission.

I moved all the HTML files away from the HTML subfolder into the main folder.

My navigation has been changed to include a button for the current page to make the navigation bars all identical. Additionally, the button of the current page is set to a different color to show which page the user is currently on.

The copyright notice in the footer of each page has been changed to be more detailed.

The CSS in the print stylesheet has been changed to set all background colours to white and remove background images.

JavaScript

Email Form

JavaScript has been used in the email form page to process and display the information that the user has input. To begin, the function creates variables with references to each input. Before continuing, the values for the required first name, last name, email, and age range values are checked to see if they are null, if any of them are an error is displayed through an alert, the data is no longer submitted and the page is no longer refreshed using the ‘event.preventDefault()’ method, and the function returns false to prevent the rest of the function from running. After this, the values of each input variable are added to a string named ‘out’. A loop is used to add the values of each checked checkbox (unless none are checked, then it adds that no sectors were chosen). When all values are added, the string is output through an alert and then the page is closed using the ‘window.close()’ method since the page is no longer needed.

JavaScript Examples

In my JavaScript Examples skill page, there are examples of JavaScript programs I have created including a rock paper scissors game and a simple calculator. The code for the JavaScript programs is not located on the website, instead they are hosted publicly on GitHub.

Additionally, the programs are displayed through an ‘iframe’ element using a URL to the hosted website in the ‘src’ attribute. JavaScript is used to handle changing the program by using buttons that have values of the URL to the corresponding website. The buttons have an ‘onclick’ attribute that calls a function in an external JavaScript file that sets the ‘src’ attribute of the iframe to the value of the button clicked.

CSS

New CSS

The CSS on my website has mostly been kept the same as my previous submission for the third assessment task.

Most of my new CSS rules are applied to the form on the new email newsletter signup page. The form was changed to a darker color to make it easier to read over the light background. I styled borders around the ‘fieldset’ elements to make it easy to distinguish between the different sections of the form. I changed the color of the required labels to make them stand out and seem more important. Finally, I used a box shadow on the form to make it look better aesthetically.

Scalability

I changed some of the previous CSS rules to make the website scale better for smaller screens.

I removed rules that used absolute measurements for container element sizes. Doing this lets the containers grow in height as needed instead of setting a maximum height that can cause issues if the text within the container needs to expand further down the page due to smaller screen width.

I added the ‘flex-wrap’ property to the horizontal navigation bars. This makes them move down the page instead of changing the scale when the screen’s width is too small.

Reusability and Readability

CSS

I only used external CSS for the website to reuse the CSS rules across the entire website instead of using internal CSS with multiple repeated rules.

This also makes the code more readable since all CSS is separated from the HTML in its own file.

Additionally, I used a lot of class selectors to change the CSS of different elements across the whole website with one rule. This made the code more reusable and more readable because it cuts down on duplicate CSS rules.

Some examples of this in my code are the use of class selectors to create identical headers, footers, and navigation bars. It also helped create the structure of the webpages using flexbox CSS rules in the body to make the main content of each page expand as far as it is required and to structure the header, main content, and footer of each page.

JavaScript

In addition to external CSS, I also used external JavaScript files to make the code more readable by separating the JavaScript from the HTML.

In the JavaScript Examples skill page, I used the 'onclick' attribute for the buttons instead of adding an event listener using the JavaScript code. This made it easier to add more examples to the page. Additionally, I used the event variable for the function to change examples. This let me easily get the value of the button pressed using the 'event.target.value' statement instead of needing to use a unique function for each button.

File Sizes and Naming

Images

To reduce the size of images used on the website, I mostly used the JPEG format which is smaller than a PNG image. The logo image is a PNG, but the dimensions are made smaller than the other images (since the image is only displayed as small on the website), allowing the file size to remain small.

Variable, Class, and ID naming

I tried to keep the names of variables and selectors short to coding easier while also keeping them descriptive to make it easier for others to read and understand my code.

Testing Plan

1. Start from index.html (home page)
2. Check content appears correctly
    a. Are CSS rules properly applied?
    b. Text formatted correctly?
    c. Images appear?

    d. Icon appears in tab?
    e. Do all elements appear properly?
3. Test if the print styles work
    a. Ctrl + p, are the print styles applied?
4. Test if footer links work
    a. Email link
    b. Phone link
    c. Email newsletter sign up link
5. Test each button in the horizontal navigation bar
    a. Does the button navigate to a new page correctly?
    b. Is the new page correct?
    c. Is the button for the current page colored differently?
6. Repeat steps 2 to 5 for the about, resume and webskills pages
7. Go to resume page and test the links to the new skills pages
    a. Do the links navigate to the correct page?
    b. Do the pages open in a new window?
    c. Repeat steps 2-4 for each skill page
8. Navigate to the JavaScript Examples page through the link in the resume page
    a. Is the iframe already displaying a program (should be the etch-a-sketch program)
    b. Does each button work? (should change the example in the iframe)
    c. Are the programs correct for each corresponding button?
9. Navigate to email form page using link in the footer of any other page
10. Input data into all fields and click submit?
    a. Does the data you input display in an alert?
    b. Does the window close?
11. Return to email form page, input data into all fields and click reset
    a. Are all input fields empty again?
12. Input data in all fields except for at least one of the first name, last name, email, or age range and click submit
    a. The data should not be submitted or output
    b. The window should stay open
    c. Red text should appear showing the required fields

Validations

HTML

Showing results for uploaded file index.html (checked with vnu 26.6.6)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only [Options…]

Check by [file upload] [Choose File] No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

[Check]

Document checking completed. No errors or warnings to show.

Showing results for RKirkland_A3_About.html (checked with vnu 26.6.6)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only [Options…]

Check by [file upload] [Choose File] No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

[Check]

Document checking completed. No errors or warnings to show.

Showing results for RKirkland_A3_Resume.html (checked with vnu 26.6.6)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only [Options…]

Check by [file upload] [Choose File] No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

[Check]

Document checking completed. No errors or warnings to show.

Showing results for RKirkland_A3_Web-Skills.html (checked with vnu 26.6.6)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only [Options…]

Check by [file upload] [Choose File] No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

[Check]

Document checking completed. No errors or warnings to show.

Showing results for RKirkland_A4_Email-Form.html (checked with vnu 26.6.6)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only [Options…]

Check by [file upload] [Choose File] No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

[Check]

Document checking completed. No errors or warnings to show.

Showing results for RKirkland_A4_Fast-Learning.html (checked with vnu 26.6.6)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only Options...

Check by file upload Choose File No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

Check

Document checking completed. No errors or warnings to show.

Showing results for RKirkland_A4_JS-Examples.html (checked with vnu 26.6.6)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only Options...

Check by file upload Choose File No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

Check

Document checking completed. No errors or warnings to show.

Showing results for RKirkland_A4_Programming-Skills.html (checked with vnu 26.6.6)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only Options...

Check by file upload Choose File No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

Check

Document checking completed. No errors or warnings to show.

Showing results for RKirkland_A4_Time-Management.html (checked with vnu 26.6.6)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only Options...

Check by file upload Choose File No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

Check

Document checking completed. No errors or warnings to show.

CSS

W3C CSS Validator results for portfolio.css (CSS level 3 + SVG)

Congratulations! No Error Found.

W3C CSS Validator results for print.css (CSS level 3 + SVG)

Congratulations! No Error Found.

Progress Table

Task	Completed?
Home Page	Yes

Resume Page	Yes
Web Skills & JavaScript Skills Page	Yes
Create other skills pages	Yes
Create form for email sign up	Yes
Make form validate if names, email, and age have been input	Yes
Make form display dat input from the user	Yes
Add reset button to form	Yes